🧠 How JavaScript Code Executes (Behind the Scenes)

✅ 1. JavaScript is single-threaded

- JS runs one line at a time → **synchronous** by default.
- Uses a **Call Stack** to manage function execution.

🔄 2. Two Phases of Code Execution

🟦 A. Memory Creation Phase (Creation Phase)

- **Global Execution Context** is created.
- **Memory is allocated** for all variables and functions:
  - `var` → initialized to `undefined`
  - `let`/`const` → created in **"Temporal Dead Zone" (TDZ)** (not initialized yet)
  - Functions → whole function is stored

🟩 B. Execution Phase

- Code is **executed line-by-line**.
- Variables get their **assigned values**
- Functions are **invoked**, and new **execution contexts** are created and pushed to the **Call Stack**.

🔁 3. Execution Context (EC)

3.1. Types:

- 🌐 **Global Execution Context (GEC)** – created for the main script

- 🧠 **Function Execution Context (FEC)** – created whenever a function is called

3.2. Each Execution Context has:

1. **Memory (Variable Environment)** → stores variables & functions
2. **Code Execution Component** → executes code line by line

🗂️ 4. Call Stack

- A **stack (LIFO)** that keeps track of function calls
- The GEC is always at the bottom

📊 **Example:**
js
CopyEdit
```
function one() {
  console.log("One");
  two();
}

function two() {
  console.log("Two");
}

one();
```

🧩 **Call Stack Order:**

1. Global Context → created
2. `one()` → pushed
3. `two()` → pushed
4. `two()` → popped after completion
5. `one()` → popped

⚠️ 5. Hoisting

- During memory phase, JS **hoists**:
  - `var` → declared & initialized as `undefined`
  - `function` → whole function is hoisted
  - `let`/`const` → hoisted but not initialized (TDZ)

🕓 6. Asynchronous Code – Event Loop (Bonus)

Not part of main stack, but interacts via:

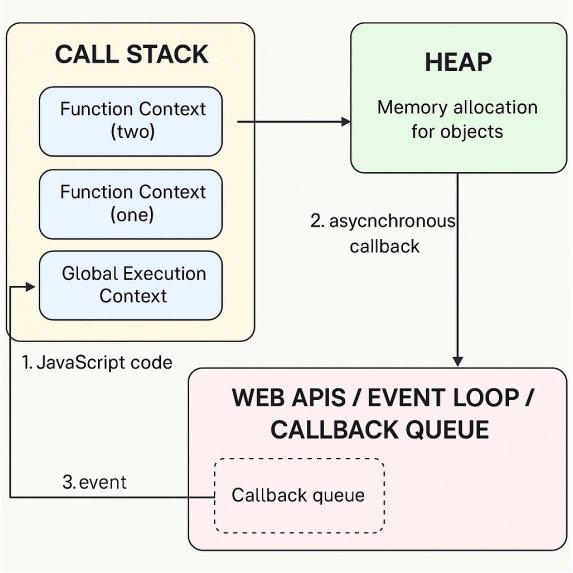
- **Web APIs (setTimeout, fetch, DOM events)**
- **Callback Queue**
- **Event Loop** → moves tasks from queue to stack when stack is empty

---

🧠 Final Mental Model

1. **JS Code** runs → Global Execution Context is created
2. Memory phase: **allocates memory**
3. Execution phase: **runs code**
4. Function calls → create new **Execution Contexts**
5. **Call Stack** manages the order
6. Asynchronous tasks are handled via **Web APIs + Event Loop**

✅ Quick Recap Table:

Concept	Description

Memory Phase	Allocates memory for variables/functions
Execution Phase	Executes code line-by-line
Execution Context	Environment in which JS code runs
Global Context	Created for the whole script
Function Context	Created each time a function is called
Call Stack	Manages execution order (LIFO)
Hoisting	Declarations moved to top in memory phase
TDZ	Temporal Dead Zone for `let` and `const`
Event Loop	Handles async code (setTimeout, fetch, etc.)